

GhostHunt

Project Proposal

Project Advisor:

Sarah Abid Khan

Group Members:

Muhammad Umer	23L-3085
Talha Farooq	23L-3043
Ahmed Javed Khan	23L-3092

National University Of Computer and Emerging Sciences
Department of Computer Science
Lahore, Pakistan

Abstract

Modern Windows systems remain vulnerable to stealth malware that evades detection by manipulating what the operating system reports to the user, without requiring kernel-level access. This project proposes GhostHunt, a user-mode rootkit detection system that identifies stealth and persistence techniques through multi-signal behavioral analysis rather than signature matching. GhostHunt correlates three independent detection signals including cross-view process enumeration, startup/persistence monitoring and DLL/driver signature verification into a single, explainable risk assessment. By combining these signals rather than relying on any one in isolation, GhostHunt aims to catch rootkit-like behavior that a single detection method would miss while explicitly scoping its guarantees to user-mode threats.

1. Introduction

Rootkits are a class of malware designed to stay hidden from standard system tools by intercepting the calls that would otherwise reveal their presence. User-mode rootkits achieve this by hooking APIs such as process listing or file enumeration and modifying what gets returned to the caller while still depending on a persistent store and an auto run mechanism to survive a reboot. Despite being a well-documented technique, most lightweight detection tools rely on a single method such as one process-listing API [4] which a rootkit built around that exact method can defeat. GhostHunt addresses this gap by never trusting a single source of truth, it independently queries multiple system views using the standard process-enumeration API [4], the lower-level native API [5] and WMI [6], so that a rootkit would have to evade all of them simultaneously to remain hidden.

2. Goals and Objectives

The goal of this project is to develop GhostHunt a multi-signal detection system that identifies user-mode rootkit and stealth-malware behavior on Windows and reports its findings with clear, evidence-based explanations.

To achieve this goal, the following objectives have been identified:

1. To detect process-hiding behavior by comparing process enumeration results from multiple independent system APIs [4] [5] [6] and flagging inconsistencies between them. To recognize user intent from natural language commands and accurately determine the task or workflow the user wishes to perform.

2. To detect unauthorized persistence by establishing a baseline of startup registry keys, scheduled tasks, and services, and identifying unapproved changes to that baseline over time.

3. To detect suspicious code-loading behavior by verifying the digital signatures of DLLs loaded into running processes [7] [8] and flagging unsigned or abnormally located modules.

4. To correlate findings across all three signals into a single weighted risk score, so that a process flagged by multiple signals is treated with higher confidence than one flagged by a single signal.

5. To validate the system against a controlled set of synthetic test cases that simulate user-mode hiding, unauthorized persistence, and unsigned code injection, without using real-world malware.

6. To measure GhostHunt's effectiveness in terms of detection rate, false-positive rate, and detection latency across all three signals.

3. Scope of the Project

The scope of this project is limited to user-mode rootkit and stealth-malware detection on the Windows operating system. GhostHunt will not attempt to detect kernel-mode rootkits since a kernel-mode attacker can control what every user-mode API reports and would therefore make all monitored views consistent with each other; defending against that threat class would require a kernel-mode driver which is outside the scope of this project.

GhostHunt will implement three detection components built in Python [1]. A process-view engine will query process lists through at least two independent methods using psutil [9] alongside the native and WMI APIs [5] [6] and flag processes present in one view but absent from another. A persistence monitor will capture a baseline of registry Run keys scheduled tasks and services, then periodically re-scan and diff against that baseline to flag new or modified entries. A signature checker will inspect DLLs loaded by running processes using pefile [8] and WinVerifyTrust [7] and flag unsigned modules or modules loaded from unexpected paths. These three signals will feed into a correlation layer that combines them into a single weighted risk score per process giving higher confidence to processes flagged by more than one component and the results will be presented through a Flask-based dashboard [2] backed by a local SQLite store [3] showing the risk-ranked process list and the evidence behind each flag.

The system will be validated using self-written simulator programs that each trigger exactly one detection signal in a controlled, isolated environment with no real malware or real rootkit techniques used at any stage. Effectiveness will be measured through detection rate, false-positive rate against normal system activity and detection latency for each signal as well as a comparison between single-signal detection and combined multi-signal detection to demonstrate the value of correlating evidence.

4. Initial Study and Work Done so Far

Rootkit behavior as covered in coursework on stealth malware falls into two categories as user-mode rootkits, which intercept API calls such as file or process listing and alter the returned results and kernel-mode rootkits, which intercept native APIs [5] and can remove a process entirely from the kernel's own process list. Documented real-world examples such as rootkits hiding a process from Windows Task Manager while it remains visible through other means, illustrate that user-mode hiding is typically partial rather than total, which is the exact inconsistency a multi-view comparison [4] [5] [6] is designed to expose. Existing lightweight anti-rootkit tools generally rely on a single detection method whereas GhostHunt's planned contribution is combining multiple independent signals specifically process-view comparison, persistence monitoring and signature verification [7] [8], into one correlated assessment rather than treating each in isolation.

References

[1] Python Software Foundation. *Python 3 Documentation*. https://docs.python.org/3/

[2] Pallets Projects. *Flask Documentation*. https://flask.palletsprojects.com/

[3] SQLite Consortium. *SQLite Documentation*. https://www.sqlite.org/docs.html

[4] Microsoft. *Process Status API (psapi.h) / Toolhelp32 Reference*. Microsoft Learn. https://learn.microsoft.com/en-us/windows/win32/toolhelp/taking-a-snapshot-and-viewing-processes

[5] Microsoft. *NtQuerySystemInformation function*. Microsoft Learn. https://learn.microsoft.com/en-us/windows/win32/api/winternl/nf-winternl-ntquerysysteminformation

[6] Microsoft. *WMI — Windows Management Instrumentation*. Microsoft Learn. https://learn.microsoft.com/en-us/windows/win32/wmisdk/wmi-start-page

[7] Microsoft. *WinVerifyTrust function (wintrust.h)*. Microsoft Learn. https://learn.microsoft.com/en-us/windows/win32/api/wintrust/nf-winverifytrust

[8] Hammond, E. *pefile — Python module to read and work with PE files*. https://github.com/erocarrera/pefile

[9] Rodola, G. *psutil — Cross-platform process and system utilities for Python*. https://psutil.readthedocs.io/